

JavaScript Data Types

JavaScript is a **dynamically typed** language — meaning you don't have to specify the type of a variable when declaring it.

The type is determined automatically when a value is assigned.

1. Primitive Data Types

Primitive data types are **basic, immutable (cannot be changed)** data types that store **a single value**.

When assigned to a variable, they hold the actual value (stored in the stack memory).

There are **7 primitive types** in JavaScript:

1.1 String

Represents a sequence of characters — text.

```
let name = "Ismail";  
let greeting = 'Hello';  
let message = `Welcome ${name}`;
```

✓ Notes:

- Strings can be in single (' '), double (" ") or backticks (` , template literals).
- Backticks allow embedding variables using `${}` .

1.2 Number

Represents both integer and floating-point numbers.

```
let age = 25;  
let price = 9.99;  
let temp = -5;
```

✓ Notes:

- JavaScript uses **64-bit floating point** representation.
- Special numeric values: `Infinity` , `-Infinity` , and `NaN` (Not-a-Number).

1.3 Boolean

Represents a logical value: `true` or `false` .

```
let isOnline = true;
let hasAccess = false;
```

✓ Used in:

Conditions, comparisons, and decision-making.

1.4 Undefined

A variable that has been declared but **not assigned a value**.

```
let color;
console.log(color); // undefined
```

✓ Notes:

- Automatically assigned by JavaScript when no value is given.

1.5 Null

Represents **intentional absence of a value**.

```
let car = null;
```

✓ Notes:

- It's an assigned value (not automatic).
- Type of `null` is "object" (this is a long-known JavaScript bug).

1.6 Symbol (ES6)

Represents a **unique and immutable** identifier.

```
let id1 = Symbol("id");
let id2 = Symbol("id");
console.log(id1 === id2); // false
```

✅ Use case:

When you need unique property keys in objects.

1.7 BigInt (ES2020)

Used for numbers **larger than** $2^{53} - 1$ (the limit for Number type).

```
let bigNum = 123456789012345678901234567890n;
```

✅ Note:

- Add `n` at the end of the number.
- BigInt cannot be mixed directly with Number types.

2. Reference Data Types

Reference types (also called **non-primitive types**) are **objects** that store multiple values.

They are **mutable** and stored by **reference**, not by value.

When assigned or passed, only the **reference (memory address)** is copied.

2.1 Object

An unordered collection of key-value pairs.

```
let person = {  
  name: "Ismail",  
  age: 30,  
  isMarried: false  
};
```

✓ Notes:

- Keys are called **properties**.
- Values can be any data type (even other objects or functions).

Accessing values:

```
console.log(person.name);    // Dot notation  
console.log(person["age"]);  // Bracket notation
```

2.2 Array

An ordered list of items (values).

```
let fruits = ["apple", "banana", "cherry"];
```

✓ Notes:

- Arrays are a special kind of object.
- Access by index:

```
console.log(fruits[0]); // apple
```

- Can store mixed types:

```
let mixed = [1, "two", true];
```

2.3 Function

A reusable block of code.

```
function greet() {  
  console.log("Hello!");  
}  
greet();
```

✓ Notes:

- Functions are **first-class objects** in JavaScript — meaning they can be stored in variables, passed as arguments, or returned from other functions.

2.4 Date

Used to work with dates and times.

```
let today = new Date();  
console.log(today);
```

2.5 Other Reference Types

- **RegExp (Regular Expressions)**
- **Map / Set / WeakMap / WeakSet**

```
let map = new Map();  
map.set("name", "Ismail");  
console.log(map.get("name")); // Ismail
```

3. Key Difference: Primitive vs Reference

Feature	Primitive	Reference
Stored in	Stack	Heap
Type	Single value	Collection or object
Mutable	No	Yes
Copied by	Value	Reference

Feature	Primitive	Reference
Examples	String, Number, Boolean	Object, Array, Function

Example: Copy Behavior

```
// Primitive
let x = 10;
let y = x;
y = 20;
console.log(x); // 10 (independent copy)

// Reference
let obj1 = { name: "Ismail" };
let obj2 = obj1;
obj2.name = "John";
console.log(obj1.name); // John (same reference)
```